

```
1  """
2  FASTQ Parser Module
3  =====
4  Parses FASTQ files using BioPython and exposes structured read data.
5  """
6  from collections import namedtuple
7  from pathlib import Path
8  from Bio import SeqIO
9
10 # Structured representation of a single FASTQ read
11 FastqRead = namedtuple("FastqRead", ["id", "sequence",
12 "quality_scores"])
13
14 def parse_fastq(filepath):
15     """
16     Generator that yields FastqRead namedtuples from a FASTQ file.
17     """
18     filepath = Path(filepath)
19     if not filepath.exists():
20         raise FileNotFoundError(f"FASTQ file not found: {filepath}")
21
22     for record in SeqIO.parse(str(filepath), "fastq"):
23         yield FastqRead( id=record.id,
24                         sequence=str(record.seq),
25                         quality_scores=record.letter_annotations["phred_quality"],
26                         )
27
28 # def count_reads(filepath):
29 #     """
30 #     Count the total number of reads in a FASTQ file.
31 #     """
32 #     count = 0
33 #     for _ in parse_fastq(filepath):
34 #         count += 1
35 #     return count
36
37 def count_reads(filepath):
38     return sum(1 for _ in parse_fastq(filepath=filepath))
39
40 def get_read_lengths(filepath):
41     """
42     Return a list of read lengths from a FASTQ file.
43     """
44     return [len(record.seq) for record in
45 parse_fastq(filepath=filepath)]
46
47
48 import sys
```

```
49
50 if __name__ == "__main__":
51     #sys.argv [fastq_parser.py, fastq_file]
52
53     if len(sys.argv) < 2:
54         print("Usage: python fastq_parser.py <fastq_file>")
55         sys.exit(1)
56
57     fq_path = sys.argv[1]
58     print(f"Parsing: {Path(fq_path).name}")
59     total = 0
60     for read in parse_fastq(fq_path):
61         total += 1
62         if total <= 3:
63             print(f"  Read {total}: {read.id}  len={len(read.sequence)}")
64     "
65
66     f"avg_qual={sum(read.quality_scores)/len(read.quality_scores):.1f}"
67     print(f"Total reads: {total}")
68
```